Day 13: Deposit Function

1. Day Number

Day 13

2. Topic Name

Functions, Parameters, and Return Values

3. Connection with Previous Learning

Earlier, you wrote deposit logic directly in your program:

- Read the current balance
- Read the deposit amount
- Check whether the amount is positive
- Update the balance

Today, you will organize this logic inside a **function**.

This makes the deposit logic:

- Easier to understand
- Easier to reuse
- Easier to test
- Easier to change later

---

4. Important Topics

`def`

The `def` keyword is used to create a function.

```
def function_name():
    # function work
```

Parameter

A parameter is a value that a function receives.

```
def show_balance(balance):
    print(balance)
```

Here, `balance` is a parameter.

`return`

The `return` keyword sends a result back to the place where the function was called.

```
def double(number):
    return number * 2
```

Function Call

A function does not run automatically after being defined. You must call it.

```
result = double(5)
```

---

5. Foundational Notes

What is a function?

A function is a reusable block of code that performs one specific task.

Think of it like a small machine:

```
Input → Function performs work → Output
```

For today's deposit function:

```
Current balance + Deposit amount
                ↓
        Deposit function
                ↓
         Updated balance
```

Defining and calling are different

Defining a function means creating it:

```
def greet():
    print("Hello")
```

Calling a function means running it:

```
greet()
```

Parameters versus arguments

In this function definition:

```
def multiply(number, value):
```

`number` and `value` are **parameters**.

In this function call:

```
multiply(5, 2)
```

`5` and `2` are **arguments**.

Why return the balance?

The function should calculate the new balance and send it back.

The returned value can then be stored:

```
updated_value = some_function(...)
```

Remember:

```
print()  → displays a value
return   → sends a value back
```

---

6. Easy Example

This example returns an updated score:

```
def add_points(score, points):
    if points > 0:
        return score + points

    return score
```

Function call:

```
new_score = add_points(10, 5)
print(new_score)
```

Output:

```
15
```

The function:

1. Receives `score` and `points`
2. Checks whether `points` is positive
3. Adds the points when valid
4. Returns the updated score
5. Returns the original score when the points are invalid

This is similar to today's deposit problem, but it is not the complete deposit solution.

---

7. Problem Statement

Create a function that accepts:

- Current balance
- Deposit amount

The function must:

- Check whether the deposit amount is positive
- Return the updated balance when the amount is positive
- Return the same balance when the amount is `0` or negative

Example behaviour

```
Balance: $500
Deposit amount: $100
Returned balance: $600
```

Invalid deposit:

```
Balance: $500
Deposit amount: -$50
Returned balance: $500
```

---

8. Concepts Used

Concept	Purpose
Function	Organizes deposit logic
`def`	Defines the function
Parameters	Receive balance and deposit amount
Comparison operator	Checks whether the amount is positive
`if` statement	Makes the deposit decision
Arithmetic operator	Adds the amount to the balance
`return`	Sends the resulting balance back
Function call	Runs the deposit function
Variable	Stores the returned balance

---

9. Thought Process

Step 1: Identify the function's responsibility

The function should perform only one main task:

> Validate the deposit amount and return the correct balance.

Step 2: Identify the required inputs

The function needs two values:

```
Current balance
Deposit amount
```

These values should become function parameters.

Step 3: Validate the amount

A valid deposit must be greater than `0`.

Think about this condition:

```
Is deposit amount greater than 0?
```

Step 4: Handle a valid deposit

When the amount is positive:

```
Updated balance = current balance + deposit amount
```

Return the updated balance.

Step 5: Handle an invalid deposit

When the amount is `0` or negative:

```
Do not change the balance
```

Return the original balance.

Step 6: Call the function

Pass the balance and deposit amount to the function.

Step 7: Store the returned value

The function's returned result should replace or update the balance variable.

---

10. Pseudocode

```
START

DEFINE a deposit function with balance and amount parameters

    IF amount is greater than zero
        Calculate balance plus amount
        RETURN updated balance
    OTHERWISE
        RETURN original balance

Create a starting balance

Ask the user for a deposit amount
Convert the deposit amount into a number

Call the deposit function
Store its returned value

Display the resulting balance

END
```

---

11. Suggested Solving Approach: Functional Approach

Divide the program into two parts.

Part 1: Deposit function

The function should:

```
Receive values
Validate the amount
Calculate the result
Return the balance
```

It should not need to ask the user for input.

Part 2: Main program

The main program should:

```
Create the balance
Ask for the deposit amount
Call the function
Save the returned balance
Print the final balance
```

This separation makes the code cleaner:

```
Main program → Handles input and output
Function     → Handles deposit logic
```

---

12. Easy Edge Cases

Deposit amount is `0`

```
Balance: $500
Deposit: $0
Result: $500
```

A zero deposit should not change the balance.

Deposit amount is negative

```
Balance: $500
Deposit: -$100
Result: $500
```

A negative deposit should not reduce the balance because this function is only for depositing money.

Positive decimal amount

```
Balance: $500
Deposit: $50.50
Result: $550.50
```

Consider using `float()` when decimal deposits are allowed.

---

13. Expected Input

Example 1:

```
Enter deposit amount: 200
```

Example 2:

```
Enter deposit amount: 0
```

Example 3:

```
Enter deposit amount: -50
```

---

14. Expected Output

For starting balance `$500` and deposit `$200`:

```
Updated balance: $700
```

For starting balance `$500` and deposit `$0`:

```
Updated balance: $500
```

For starting balance `$500` and deposit `-$50`:

```
Updated balance: $500
```

---

15. Hint Only

Start with this incomplete structure:

```
def deposit(balance, amount):
    # Check whether amount is positive

    # Return updated balance for a valid amount

    # Return original balance for an invalid amount
```

Remember to store the returned result:

```
balance = deposit(balance, deposit_amount)
```

The most important condition is:

```
amount must be greater than 0
```